

LAPORAN PRAKTIKUM PEKAN 1
PEMOGRAMAN BERORIENTASI OBJEK

Class, Object, Constructor, dan Method
Interaksi Objek & Manipulasi State (Mini Bank)



Oleh:
Naysha Aprilia Rizki (2511531014)
Kelas B

Dosen Pengampu:
Nurfiah, S.ST., M.Kom.

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
PADANG
2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan Laporan Praktikum Pekan 1 mata kuliah Pemrograman Berorientasi Objek dengan materi Class, Object, Constructor, dan Method serta Interaksi Objek dan Manipulasi State (Mini Bank) dengan baik.

Laporan ini disusun sebagai bentuk dokumentasi dan pertanggungjawaban atas pelaksanaan praktikum yang telah dilakukan. Melalui praktikum ini, penulis mempelajari konsep dasar pemrograman berorientasi objek, khususnya mengenai class, object, constructor, dan method, serta penerapannya dalam membangun program sederhana berupa Mini Bank. Praktikum ini juga membantu penulis memahami bagaimana objek dapat saling berinteraksi dan bagaimana state suatu objek dapat dimanipulasi melalui method.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dalam penyusunan maupun penyajian materi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan evaluasi dan perbaikan untuk laporan maupun praktikum selanjutnya.

Akhir kata, penulis berharap laporan ini dapat memberikan manfaat dan menambah pemahaman mengenai konsep dasar Pemrograman Berorientasi Objek, khususnya dalam penerapan class, object, constructor, method, serta interaksi dan manipulasi state pada sebuah program.

Padang, 8 September 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pemrograman merupakan salah satu bagian penting dalam ilmu komputer yang digunakan untuk mengembangkan berbagai aplikasi dan sistem. Salah satu pendekatan pemrograman yang banyak digunakan adalah Pemrograman Berorientasi Objek (PBO). Pemrograman Berorientasi Objek merupakan paradigma pemrograman yang menggunakan objek sebagai dasar dalam membangun sebuah program. Dengan pendekatan ini, program dapat dibuat lebih terstruktur, mudah dikembangkan, dan lebih mudah dipahami.

Dalam Pemrograman Berorientasi Objek, terdapat beberapa konsep dasar yang perlu dipahami, seperti class, object, constructor, dan method. Class digunakan sebagai rancangan atau cetak biru untuk membentuk suatu objek, sedangkan object merupakan bentuk nyata dari class yang memiliki atribut dan perilaku tertentu. Constructor digunakan untuk melakukan inisialisasi objek ketika objek dibuat, sementara method digunakan untuk mendefinisikan tindakan atau perilaku yang dapat dilakukan oleh suatu objek.

Selain memahami konsep dasar tersebut, interaksi antarobjek dan manipulasi state juga merupakan bagian penting dalam Pemrograman Berorientasi Objek. State merupakan kondisi atau nilai yang dimiliki oleh suatu objek dan dapat berubah melalui method tertentu. Penerapan konsep tersebut dapat digambarkan melalui pembuatan program sederhana seperti Mini Bank, di mana objek dapat memiliki data rekening, melakukan transaksi, serta mengubah keadaan atau nilai yang dimilikinya.

Oleh karena itu, praktikum ini dilakukan agar mahasiswa dapat memahami konsep dasar Pemrograman Berorientasi Objek, khususnya class, object, constructor, dan method, serta mampu menerapkan konsep interaksi objek dan manipulasi state dalam bahasa pemrograman Java melalui pembuatan program Mini Bank.

1.2 Tujuan

1. Mendefinisikan class sebagai model dari entitas dunia nyata
2. Mendefinisikan object dan memahami alokasi memori independent
3. Menggunakan constructor untuk inisialisasi awal state
4. Manipulasi state (atribut) melalui behavior (method) menggunakan interaksi CLI

1.3 Manfaat

1. Memahami konsep class sebagai model atau rancangan dari suatu entitas dunia nyata sehingga dapat digunakan sebagai dasar dalam pembuatan program berorientasi objek.
2. Memahami konsep object serta mengetahui bahwa setiap object memiliki alokasi memori yang independent, sehingga setiap object dapat memiliki state atau nilai atribut yang berbeda.
3. Memahami penggunaan constructor untuk memberikan nilai awal atau melakukan inisialisasi state pada saat object dibuat.
4. Memahami manipulasi state melalui method serta mengetahui bagaimana suatu object dapat menjalankan behavior tertentu melalui interaksi berbasis Command Line Interface (CLI).
5. Meningkatkan kemampuan dalam menerapkan konsep Pemrograman Berorientasi Objek melalui pembuatan program sederhana yang menggambarkan interaksi antarobject dan perubahan state.

BAB II

PEMBAHASAN

2.1 Teori

Dalam Pemrograman Berorientasi Objek (PBO), perangkat lunak dimodelkan berdasarkan entitas di dunia nyata. Dua pilar paling mendasar dalam pemodelan ini adalah Class dan Object.

- **Class sebagai Blueprint**, Menurut Deitel & Deitel (2017) dalam bukunya Java: How to Program, sebuah class adalah cetak biru (blueprint) dari sebuah objek. Class mendefinisikan struktur data (attributes/state) dan sekumpulan operasi (methods/behavior) yang dapat dilakukan pada data tersebut. Class sendiri tidak memakan ruang memori untuk menyimpan nilai data, melainkan hanya berfungsi sebagai tipe data referensi.
- **Object dan State**, Objek adalah instansiasi konkret dari sebuah class yang dialokasikan di dalam memori komputer. Setiap objek memiliki state/kondisi yang independen. Sierra & Bates (2005) dalam Head First Java mengilustrasikan bahwa dua objek dari class yang sama misalnya Rekening dapat memiliki nilai state seperti saldo yang sama sekali berbeda, dan perubahan pada satu objek tidak akan memengaruhi objek lainnya.
- **Constructor** adalah blok kode khusus yang dieksekusi secara otomatis pada saat objek dibuat menggunakan keyword new. Fungsinya adalah untuk menginisialisasi state awal dari objek tersebut (Oracle, 2023).

Dalam konteks perbankan, Class adalah konsep "Rekening", Object adalah "Rekening milik Mawar" atau "Rekening milik Melati", State adalah "Saldo saat ini", dan Behavior adalah aktivitas seperti "Setor Tunai" yang akan memanipulasi state saldo tersebut.

2.2 Langkah Langkah

1. Membuat Class Rekening

- 1) Pertama saya membuat Package "MiniBank"
- 2) Selanjutnya dilanjutkan dengan membuat Class Rekening, Berikut Screenshot lengkap nya

```

1 package MiniBank;
2
3 public class Rekening {
4     String nomorRekening;
5     String namaPemilik;
6     double saldo;
7
8     public Rekening(String nomor, String nama, double saldoAwal) {
9         nomorRekening = nomor;
10        namaPemilik = nama;
11        saldo = saldoAwal;
12        System.out.println("Rekening atas nama " + namaPemilik + " Berhasil dibuat dengan saldo Rp " + saldo);
13    }
14
15    public void setTunai(double nominal) {
16        if (nominal > 0) {
17            saldo += nominal;
18            System.out.println("Setor tunai Rp " + nominal + " berhasil Saldo saat ini: Rp " + saldo);
19        } else {
20            System.out.println("Gagal: Nominal setor harus lebih dari 0");
21        }
22    }
23
24    public void tampilkan() {
25        System.out.println("--- INFO REKENING ---");
26        System.out.println("No. Rekening : " + nomorRekening);
27        System.out.println("Nama Pemilik : " + namaPemilik);
28        System.out.println("Saldo Akhir : Rp " + saldo);
29        System.out.println("-----");
30    }
31 }
32

```

Penjelasan :

1) Buat class Rekening

```
public class Rekening {
```

Ini mengartikan kita sedang membuat sebuah class bernama Rekening. Class bisa dianggap sebagai cetakan/model untuk membuat objek rekening.

2) Menambahkan State/Atribut

```
String nomorRekening;
String namaPemilik;
double saldo;
```

Atribut ini merupakan variable yang menyimpan informasi tentang suatu objek. Pada class rekening ini tersimpan data berupa

- nomor rekening dengan tipe data string
- nama pemilik dengan tipe data string
- saldo dengan tipe data double.

3) Buat Constructor

```

public Rekening(String nomor, String nama, double saldoAwal) {
    nomorRekening = nomor;
    namaPemilik = nama;
    saldo = saldoAwal;
    System.out.println("Rekening atas nama " + namaPemilik + " Berhasil dibuat dengan saldo Rp " + saldo);
}

```

Constructor ini digunakan untuk menginisialisasikan atau memberi nilai awal pada objek.

- nomorRekening di inisialisasikan dengan nomor
- namaPemilik dengan nama
- saldo dengan saldo awal.
- Program akan menghasilkan output “Rekening atas nama” (berisi nama dari pemilik) Berhasil dibuat dengan saldo Rp (berisi nominal saldo dari pemilik rekening).

4) Buat Method/Behavior

```
15 public void setorTunai(double nominal) {
16     if (nominal > 0) {
17         saldo += nominal;
18         System.out.println("Setor tunai Rp" + nominal + " Berhasil Saldo saat ini: Rp" + saldo);
19     } else {
20         System.out.println("Gagal: Nominal setor harus lebih dari 0!");
21     }
22 }
23
24 public void cekInformasi() {
25     System.out.println("--- INFO REKENING ---");
26     System.out.println("No. Rekening : " + nomorRekening);
27     System.out.println("Nama Pemilik : " + namaPemilik);
28     System.out.println("Saldo Akhir : Rp" + saldo);
29     System.out.println("-----");
30 }
31 }
```

Dibagian sini Untuk Methodnya baru ada setor tunai dan cek informasi.

- Method Setor tunai itu menjelaskan “Jika nominal nya besar dari 0 maka saldo akan bertambah sebanyak nominal dan program akan mengoutputkan “Setor tunai Rp (jumlah nominal) Berhasil, saldo saat ini Rp(jumlah saldo).
- Method cek informasi. Itu mengoutputkan Info Rekening yang dimiliki Sistem mengeluarkan Nomor Rekening, Nama Pemilik, dan Juga Saldo akhir setelah dilakukannya setor tunai.

2. Membuat Class Main

Agar dapat menjalankan Class rekening tadi diperlukan Class Main. Berikut Screenshot lengkapnya.

```
1 package Rekening;
2 import java.util.Scanner;
3
4 public class Main {
5     public static void main(String[] args) {
6         Scanner input = new Scanner(System.in);
7         Rekening rekening = new Rekening(123456789, "John Doe", 1000000);
8         rekening.setSaldo(1000000);
9
10        System.out.println("===== SISTEM REKENING KAMI =====");
11
12        while (true) {
13            System.out.println("Menu Utama:");
14            System.out.println("1. Saldo Rekening Saat Ini");
15            System.out.println("2. Saldo Awal");
16            System.out.println("3. Saldo Akhir");
17            System.out.println("4. Info Rekening Rekening");
18            System.out.println("5. Saldo");
19            System.out.println("6. Saldo");
20
21            int pilihan = input.nextInt();
22            input.nextLine(); //consume newline buffer
23
24            switch (pilihan) {
25                case 1:
26                    System.out.println("Saldo Rekening: ");
27                    String no = input.nextLine();
28                    System.out.println("Nomor Saldo Rekening: ");
29                    String name = input.nextLine();
30                    System.out.println("Nomor Saldo Rekening: ");
31                    String saldo = input.nextLine();
32
33                    //Saldo Rekening (Rekening)
34                    rekening = new Rekening(no, name, saldo);
35
36                case 2:
37                    if (rekening == null) {
38                        System.out.println("Nomor Saldo Rekening, dan Saldo Rekening sama dengan 0");
39                    } else {
40                        System.out.println("Nomor Saldo Rekening: ");
41                        rekening.setSaldo(rekening.getSaldo());
42                        rekening.setSaldo(rekening.getSaldo());
43                    }
44
45                case 3:
46                    System.out.println("Info Saldo Rekening: ");
47
48            }
49        }
50    }
51 }
```

```

53
54
55     case 4:
56         if (akunAktif == null) {
57             System.out.println("Error: Anda belum memulai running!");
58         } else {
59             akunAktif.cekInformasi();
60         }
61         break;
62
63     case 0:
64         isRunning = false;
65         System.out.println("Sistem ditutup. Terima Kasih!");
66         break;
67
68     default:
69         System.out.println("Pilihkan tipe valid!");
70     }
71 }
72
73 input.close();
74
75 }

```

Penjelasan :

- 1) Import Java.util.scanner;

```
import java.util.Scanner;
```

Digunakan untuk mengimpor class Scanner yang berfungsi menerima input dari pengguna melalui keyboard.

- 2) Deklarasi Class Main

```

public class Main {
    public static void main(String[]args) {
        Scanner input = new Scanner(System.in);
        Rekening akunAktif = null; //null karna objek belum di inisialisasi
        boolean isRunning = true;
    }
}

```

- **public static void main(String[]args) {** adalah method utama yang pertama kali dijalankan ketika program Java dijalankan.
- **Scanner input = new Scanner(System.in);** adalah deklarasi dan inisialisasi objek Scanner.
- **Rekening akunAktif = null;** adalah deklarasi variabel referensi objek Rekening, yang awalnya bernilai null (objek belum di insialisasikan)
- **boolean isRunning = true;** adalah deklarasi dan inisialisasi variabel boolean untuk mengontrol program.

- 3) Menampilkan Judul Program

```
System.out.println("=== SISTEM PERBANKAN MINI ===");
```

Dari sini akan muncul judul dari program yang sedang kita buat.

- 4) Perulangan Menu utama (while)

```

while (isRunning) {
    System.out.println("\nMenu Utama:");
    System.out.println("1.Buka Rekening Baru");
    System.out.println("2.Setor Tunai");
    System.out.println("3.Tarik tunai");
    System.out.println("4.Cek Informasi Rekening");
    System.out.println("0.Keluar");
    System.out.println("Pilih Menu: ");
}

```

- **while (isRunning) {** Digunakan untuk menjalankan menu utama secara berulang selama kondisi isRunning bernilai true,

dan akan berhenti atau keluar saat memilih 0 karna itu mengubah nilai `isRunning` menjadi `false`.

- Setiap **`System.out.println`** menjelaskan apa saja menu yang akan muncul contoh :
 1. Buka Rekening, (Pilihan untuk membuka rekening)
 2. Setor tunai (pilihan untuk menyetor saldo)
 3. Tarik tunai (pilihan untuk menarik saldo)
 4. Cek Informasi (pilihan untuk melihat informasi rekening)
 0. Keluar (pilihan untuk akhiri program)

Pilih Menu (Meminta user memasukkan pemilihan menu)

5) Menerima pilihan user

```
int pilihan = input.nextInt();
input.nextLine(); //Membersihkan buffer enter
```

- **`int pilihan = input.nextInt();`** digunakan untuk menerima input berupa bilangan bulat dari user dan menyimpannya dalam variable `pilihan`
- **`input.nextLine();`** untuk membersihkan buffer enter

6) Menentukan menu pilihan user (switch-case)

```
switch(pilihan) {
    case 1:
        System.out.print("Masukan No Rekening: ");
        String no = input.nextLine();
        System.out.print("Masukan Nama Pemilik: ");
        String nama = input.nextLine();
        System.out.print("Masukan Saldo Awal: ");
        double saldo = input.nextDouble();

        //Inisialisasi Object / Menjalankan Constructor
        akunAktif = new Rekening(no, nama, saldo);
        break;

    case 2:
        if (akunAktif == null) {
            System.out.println("Error: Maaf, Anda belum memiliki nomor rekening.");
        } else {
            System.out.print("Masukan nominal setor: ");
            double setor = input.nextDouble();
            akunAktif.setorTunai(setor); //Membanggil Method
        }
        break;

    case 3:
        System.out.println("Fitur ini akan dikerjakan sebagai tugas mandiri");
        break;

    case 4:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum membuka rekening!");
        } else {
            akunAktif.cekInformasi();
        }
        break;

    case 0:
        isRunning = false;
        System.out.println("Sistem ditutup. Terima kasih!");
        break;

    default:
        System.out.println("Pilihan tidak valid!");
}
input.close();
}
```

- **switch(pilihan)** { digunakan untuk memilih dan menjalankan salah satu blok program berdasarkan pilihan user. Pilihan ini berdasarkan dari menu utama yang sudah di deklarasikan pada class Rekening sebelumnya.
- **Case 1** Bagian User membuka rekening
Berisikan data yang diminta Masuka No rekening, Masukan Nama Pemilik, Masukan saldo awal. Setelah 3 Data diinput maka akun akan aktif dan baru bisa dilanjut ke Case selanjutnya
- **Case 2** Bagian Setor Tunai
Bagian ini di gunakan agar user dapat melakukan setor tunai.
 - jika user belum memiliki akun yang aktif maka program akan mengeluarkan error karna akun belum terdeteksi. Dan akan dilakukan perulangan dan user harus melakukan case 1 terlebih dahulu.
 - Jika user sudah memiliki akun aktif maka user dapat melakukan setor tunai berdasarkan yang sudah di deklarasi pada class Rekening
- **Case 3** (Penugasan Mandiri)
Bagian ini di jadikan penugasan pribadi jadi untuk sekarang informasi yang di hasilkan jika menginputkan Case 3, Output yang akan keluar hanya “Fitur ini akan dikerjakan sebagai tugas mandiri”
- **Case 4** Bagian Cek informasi rekening
Digunakan untuk menampilkan Informasi rekening
 - jika user belum memiliki akun yang aktif maka program akan mengeluarkan error karna akun belum terdeteksi. Dan akan dilakukan perulangan dan user harus melakukan case 1 terlebih dahulu.
 - Jika user sudah memiliki akun aktif maka user dapat melihat informasi terkait rekening mereka.
- **Case 0** Keluar
Digunakan untuk mengakhiri program. Nilai yang di hitung **isRunning = false**; membuat kondisi while jadi false dan perulangan berhenti.
- **Default**
Digunakan apanila user memasukan menu yang tidak ada.
- **input.close();** Digunakan untuk menutup objek scanner setelah input dari user selesai.

2.3 Latihan/Tugas

1. Implementasi Fitur Tarik Tunai

- Tambahkan method **tarikTunai(double nominal)** ke class rekening. Dengan aturan
 - a. Saldo tidak boleh minus dan minimal Tarik tunai adalah 10.000
 - b. Jika user menarik uang lebih dari saldo saat ini atau kecil dari 10.000 maka akan menampilkan "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau "Transaksi Gagal : Minimal nomonal penarikan 10.000".
 - c. Terapkan method pada menu nomor 3 pada class main.

Method tarikTunai(double nominal) pada class rekening

```
public void tarikTunai(double nominal) {  
    if (nominal < 10000) {  
        System.out.println("Transaksi Gagal: Minimal Penarikan 10.000");  
    } else if (nominal > saldo) {  
        System.out.println("Transaksi Gagal saldo tidak mencukupi.Saldo Anda: Rp" + saldo);  
    }  
    else {  
        saldo -= nominal;  
        System.out.println("Transaksi Berhasil");  
    }  
}
```

Penjelasan:

Pada Method Tarik tunai ini di sebutkan

- Jika nominal penarikan kecil dari 10.000 maka system akan mengoutputkan "Transaksi Gagal: Minimal Penarikan 10.000"
- Jika nominal penarikan besar dari saldo user maka system akan mengoutputkan "Transaksi Gagal saldo tidak mencukupi.Saldo Anda: Rp"
- Namun jika selain itu, seperti saldo kecil sama dengan nominal dan syarat lainnya terpenuhi maka system akan mengoutputkan "Transaksi Berhasil"

Selanjutnya, tambahkan method tarikTunai ini pada Class main di bagian Switch case 3

```
case 3:  
    if (akunAktif == null) {  
        System.out.println("Error: Mohon Maaf, Anda belum memiliki nomor rekening!");  
    } else {  
        System.out.print("Masukan nominal tarik: ");  
        double tarik = input.nextDouble();  
        akunAktif.tarikTunai(tarik); //Memanggil Method  
    }  
    break;
```

Penjelasan:

Pada Case 3

- Jika akun aktif nya tidak ada/ null maka program akan mengoutputkna "Error: Mohon Maaf, Anda belum memiliki nomor rekening!"
- Jika akun sudah ada, program akan mengoutputkan "Masukan Nominal Tarik:" dan system akan memproses dan memanggil method yang sudah di deklarasikan pada class rekening tadi.

OUTPUT

```

Main (1) [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe (Sep 11, 2024)
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 100
Masukan Nama Pemilik: nay
Masukan Saldo Awal: 20.000

```

(Buka Rekening baru)

```

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
0.Keluar
Pilih Menu:
3
Masukan nominal tarik: 2000
Transaksi Gagal: Minimal Penarikan 10.000

```

(Transaksi Gagal Minimal penarikan !0.000)

```

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
0.Keluar
Pilih Menu:
3
Masukan nominal tarik: 30.000
Transaksi Gagal saldo tidak mencukupi.Saldo Anda: Rp20000.0

```

(Transaksi gagal saldo tidak mencukupi)

```
Masukan nominal tarik: 15.000
Transaksi Berhasil

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
0.Keluar
Pilih Menu:
```

(Transaksi Berhasil)

2. Tantangan Multi-Akun

Membuat multi akun dengan intruksi

- Ubah arsitektur penyimpanan di Main.java menggunakan struktur data Collection (Gunakan ArrayList<Rekening>)
- Ubah logika menu “Buka Rekening”. Setia pada user yang membuka rekening baru objek itu tambahkan ke arraylist.
- Tambahkan menu baru Ganti akun, dimana system meminta nomor rekening kemudian mencari objek yang cocok pada arraylist untuk dijadikan akun aktif

ArrayList digunakan untuk menyimpan beberapa objek Rekening dalam satu tempat. Dengan demikian, setiap rekening yang dibuat oleh pengguna dapat disimpan dan tetap dapat diakses kembali tanpa menghapus rekening sebelumnya. Pada Class Main :

- 1) Tambahkan **import java.util ArrayList**, Digunakan untuk mengimpor class Scanner yang berfungsi menerima input dari pengguna melalui keyboard.

```
import java.util.ArrayList;
```

- 2) Deklarasikan pada public class main **ArrayList<Rekening> daftarRekening = new ArrayList<>();**

```
ArrayList<Rekening> daftarRekening = new ArrayList<>();
```

Digunakan untuk menyimpan data data rekening dan daftar rekening sebagai tempat penyimpanan nya.

- 3) Instialisasi objek pada **case 1** untuk menjalankan Constructor

```
//Instansiasi Object /Menjalankan Constructor
Rekening rekeningBaru = new Rekening(no, nama, saldo);
daftarRekening.add(rekeningBaru);
akunAktif = rekeningBaru;
break;
```

- **Rekening rekeningBaru = new Rekening(no, nama, saldo);**
Diminta user meng inputkan rekening baru terlebih dahulu
- **daftarRekening.add(rekeningBaru);**
Kemudian rekening akan masuk ke daftar rekening ini
- **akunAktif = rekeningBaru;**
Akun yang aktif adalah akun yang baru di buat

4) Fitur Ganti Akun

- Pada bagian while atau perulangan tambahkan **System.out.println("5.Ganti Akun");**
- Buat Case 5 Bagian untuk Ganti akun

```
case 5:
    System.out.print("Masukan No Rekening: ");
    String cariNo = input.nextLine();
    boolean ditemukan = false;

    for (Rekening rekening : daftarRekening) {
        if (rekening.getNomorRekening().equals(cariNo)) {
            akunAktif = rekening;
            ditemukan = true;
            System.out.println("Akun Berhasil Diganti.");
            break;
        }
    }

    if (!ditemukan) {
        System.out.println("Rekening tidak ditemukan.");
    }
    break;
```

- **System.out.print("Masukan No Rekening: ");**
String cariNo = input.nextLine();
Digunakan untuk meminta nomor rekening User
- **boolean ditemukan = false;** Digunakan sebagai variable penanda apakah rekening yang dicari sudah di temukan atau belum, nilai awal fase karna rekening belum ditemukan.
- **for (Rekening rekening : daftarRekening) {** Bagian yang digunakan untuk mengecek objek rekening yang tersimpan di daftar rekening
- **if (rekening.getNomorRekening().equals(cariNo)) {** Bagian untuk membandingkan nomor rekening yang tersimpan dengan rekening yang dimasukan user
- **akunAktif = rekening;** Jika rekening ditemukan maka akan jadi akun aktif

- **ditemukan = true;** Mengubah status false diawal menjadi true karna rekening sudah ditemukan.
- **System.out.println("Akun Berhasil Diganti.");** Menampilkan output bahwa pergantian akun sudah berhasil
- **break;** digunakan untuk menghentikan perulangan for setelah rekening ditemukan.
- **if (!ditemukan) {**
 System.out.println("Rekening tidak ditemukan."); Jika memang tidak ditemukan program akan mengoutputkan "Rekening tidak ditemukan"

c. Masukkan deklarasi Constructor baru pada class Rekening

```
public Rekening(String nomorRekening, String namaPemilik, double saldo) {
    this.nomorRekening = nomorRekening;
    this.namaPemilik = namaPemilik;
    this.saldo = saldo;
}
```

Constructor ini digunakan untuk memberi nilai awal pada objek rekening, agar dapat dijalankan pada bagian Class Main yang sudah di tambahkan tadi

d. Tambahkan Method getter pada Class Rekening

```
public String getNomorRekening() {
    return nomorRekening;
}
```

Method ini gunakan untuk mengambil atau mendapatkan nilai nomor rekening pada objek rekening. Getter disini digunakan pada fitur Ganti Akun

If (rekening.getNomorRekening().equals(cariNo)) { yang mengartikan program ingin mengambil semua objek dalam daftar rekening

OUTPUT

```
Console X
Main (1) [Java Application] C:\Program Files\Java\jdk-24\bin\javaw.exe (Sep 11, 2026, 3:18:0
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 001
Masukan Nama Pemilik: nay
Masukan Saldo Awal: 200.000

Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
0.Keluar
Pilih Menu:
1
Masukan No Rekening: 002
Masukan Nama Pemilik: april
Masukan Saldo Awal: 500.000
```

(Membuat banyak Rekening dan dapat tersimpan di daftar rekening tanpa terhapus)

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
0.Keluar
Pilih Menu:
5
Masukan No Rekening: 002
Akun Berhasil Diganti.
```

(Mengganti akun berhasil dilakukan,karna nomor rekening ditemukan)

```
Menu Utama:
1.Buka Rekening Baru
2.Setor Tunai
3.Tarik tunai
4.Cek Informasi Rekening
5.Ganti Akun
0.Keluar
Pilih Menu:
5
Masukan No Rekening: 003
Rekening tidak ditemukan.
```

(Gagal karena rekening tidak ditemukan)

